



FACULTAD DE INGENIERÍA

Sistemas multiagente

Obligatorio 2: Juegos alternados.

2026

Profesor:

Marcos Adrian Begerez Oyhenard

Martín Iturbide Noria

Alumnos:

Natalia Campiglia - 349251

Marcos Mussio - 362839

Índice

1. Introducción	4
2. Marco Teórico	5
2.1. Juegos Alternados	5
2.2. Juegos de Suma Cero	5
2.3. Árboles de Juego	5
2.4. Estados de Información	6
2.5. Minimax	6
2.6. Monte Carlo Tree Search	6
2.7. Information Set Monte Carlo Tree Search	7
2.8. Counterfactual Regret Minimization	7
3. Ambientes	8
3.1. Tic-Tac-Toe	8
3.2. Nocco Nocco	9
3.3. Kuhn Poker	9
4. Implementación	10
4.1. Interfaz de Juegos Alternados	10
4.2. Agente Aleatorio	10
4.3. Minimax	10
4.4. MCTS	10
4.5. ISMCTS	11
4.6. CFR	11
5. Experimentos	12
5.1. Tic-Tac-Toe	12
5.2. Nocco Nocco	20

5.3. Leduc Hold'em	26
5.4. Kuhn Poker	32
6. Conclusión	33

1. Introducción

Los sistemas multiagente permiten modelar problemas donde varios agentes toman decisiones dentro de un ambiente compartido [5, 4]. En el Obligatorio 2 se trabaja con juegos alternados, es decir, ambientes en los que los agentes actúan por turnos y el estado del juego evoluciona luego de cada acción. Este tipo de interacción aparece en juegos de tablero, juegos de cartas y problemas secuenciales de decisión estratégica.

A diferencia de los juegos simultáneos, en los juegos alternados cada decisión puede depender del historial previo. Esto permite estudiar algoritmos de búsqueda, planificación y aprendizaje en juegos extensivos, tanto con información perfecta como con información imperfecta. En particular, este trabajo considera ambientes como Tic-Tac-Toe, Nocca Nocca y Kuhn Poker.

El objetivo del trabajo es implementar y analizar agentes capaces de jugar en estos ambientes. Para ello se desarrollaron agentes basados en Minimax, Monte Carlo Tree Search (MCTS), Information Set Monte Carlo Tree Search (ISMCTS), Counterfactual Regret Minimization (CFR) y un agente aleatorio de referencia.

El código del obligatorio se organiza alrededor de una abstracción común para juegos alternados, compatible con la interfaz de PettingZoo. Cada ambiente define acciones disponibles, observaciones, transiciones, recompensas y condiciones de finalización, mientras que los agentes consumen esa interfaz para seleccionar acciones.

El informe se organiza de la siguiente manera. Primero se presenta el marco teórico necesario para comprender juegos alternados, búsqueda adversarial y juegos con información imperfecta. Luego se describen los algoritmos implementados y los ambientes utilizados. Finalmente, se deja planteada la estructura de experimentos a completar con los resultados obtenidos.

2. Marco Teórico

2.1. Juegos Alternados

Un juego alternado es un juego secuencial en el que, en cada instante, un único agente tiene el turno de actuar. Luego de seleccionar una acción, el ambiente actualiza su estado, puede entregar recompensas y selecciona el siguiente agente. Esta dinámica se puede representar mediante un árbol de juego, donde los nodos corresponden a estados y las aristas a acciones.

En juegos de información perfecta, todos los agentes observan completamente el estado relevante del juego. Tic-Tac-Toe y Nocka Nocka pertenecen a esta familia. En juegos de información imperfecta, los agentes solo observan una parte del estado real. Kuhn Poker es un ejemplo clásico, porque cada jugador conoce su carta, pero no la carta del oponente.

2.2. Juegos de Suma Cero

Muchos juegos alternados competitivos se modelan como juegos de suma cero. En estos juegos, la ganancia de un agente implica una pérdida equivalente para el otro. Esta propiedad permite razonar en términos de maximización para el agente propio y minimización para el adversario.

2.3. Árboles de Juego

El árbol de juego representa todas las posibles continuaciones desde un estado inicial. Cada nodo contiene un estado, el agente que debe actuar y las acciones legales disponibles. Los nodos terminales representan estados finales y contienen las recompensas para cada agente.

En juegos pequeños, el árbol completo puede recorrerse de forma exhaustiva. Sin embargo, en juegos con mayor factor de ramificación o profundidad, recorrer todo el árbol resulta costoso, por lo que se usan heurísticas, límites de profundidad o métodos de muestreo.

2.4. Estados de Información

En juegos con información imperfecta, un agente no siempre puede distinguir entre todos los estados reales posibles. El conjunto de estados compatibles con lo que el agente observó se denomina estado de información. Los algoritmos para estos juegos deben decidir en función de ese conjunto, no únicamente de un estado real completamente observable.

Kuhn Poker introduce esta dificultad de forma mínima: cada jugador observa su carta privada y el historial público de apuestas, pero desconoce la carta privada del rival. Por eso resulta adecuado para evaluar algoritmos como ISMCTS y CFR.

2.5. Minimax

Minimax es un algoritmo clásico de búsqueda adversarial para juegos de dos jugadores, alternados, deterministas y de suma cero. La idea central es elegir la acción que maximiza la utilidad del agente propio, suponiendo que el oponente elegirá acciones que minimizan dicha utilidad.

La recurrencia puede expresarse como:

$$V(s) = \begin{cases} U(s), & \text{si } s \text{ es terminal} \\ \max_{a \in A(s)} V(T(s, a)), & \text{si actúa el agente propio} \\ \min_{a \in A(s)} V(T(s, a)), & \text{si actúa el oponente} \end{cases}$$

Cuando el árbol es demasiado grande, se puede limitar la profundidad de búsqueda y utilizar una función de evaluación heurística para estimar el valor de estados no terminales.

2.6. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) es un algoritmo de búsqueda que estima el valor de las acciones mediante simulaciones [1]. Cada iteración recorre cuatro etapas: selección, expansión, simu-

lación y retropropagación. La selección suele guiarse mediante UCB/UCT, balanceando explotación de acciones prometedoras y exploración de acciones poco visitadas.

MCTS resulta útil cuando el espacio de estados es grande y no se desea recorrer el árbol completo. En este obligatorio se usa como agente general para juegos alternados de información perfecta, realizando rollouts aleatorios desde estados expandidos.

2.7. Information Set Monte Carlo Tree Search

Information Set Monte Carlo Tree Search (ISMCTS) adapta MCTS a juegos con información imperfecta. En lugar de construir el árbol sobre estados reales completamente conocidos, el algoritmo opera sobre estados de información. En cada simulación se samplea una determinización compatible con lo observado por el agente y se ejecuta una búsqueda sobre ese posible estado.

Esta aproximación permite aplicar búsqueda Monte Carlo en juegos como Kuhn Poker, donde el agente debe razonar sobre cartas ocultas y decisiones del oponente sin conocer completamente el estado real.

2.8. Counterfactual Regret Minimization

Counterfactual Regret Minimization (CFR) es un algoritmo utilizado para aproximar estrategias en juegos extensivos con información imperfecta [3]. El algoritmo actualiza regrets por estado de información, comparando la utilidad esperada de las acciones disponibles contra la utilidad de la política actual.

La política se deriva de los regrets positivos acumulados mediante regret matching. Con suficientes iteraciones, en ciertos juegos de dos jugadores y suma cero, la estrategia promedio puede aproximarse a un equilibrio de Nash.

3. Ambientes

3.1. Tic-Tac-Toe

Tic-Tac-Toe es un juego alternado, determinista, de suma cero e información perfecta para dos jugadores. El tablero tiene dimensión 3×3 y los jugadores colocan marcas por turnos. El objetivo es completar una fila, columna o diagonal con tres marcas propias antes que el oponente.

En cada turno, el agente activo elige una acción entre las posiciones vacías del tablero. Como el tablero tiene nueve celdas, al inicio existen nueve acciones posibles. Luego de cada movimiento, el conjunto de acciones legales se reduce a las casillas todavía libres. La observación del agente contiene el estado completo del tablero, por lo que no existe información oculta ni incertidumbre sobre las jugadas anteriores.

Los valores terminales posibles para un agente son:

$$R_i = \begin{cases} 1, & \text{si el agente gana la partida} \\ -1, & \text{si el agente pierde la partida} \\ 0, & \text{si la partida termina en empate} \end{cases}$$

Como es un juego de suma cero, la recompensa del oponente es la opuesta cuando hay ganador. Si la partida termina en empate, ambos agentes reciben recompensa cero. En la implementación utilizada, estas recompensas se obtienen desde el ambiente al alcanzar un estado terminal.

Desde el punto de vista teórico, Tic-Tac-Toe es un juego resuelto. Si ambos jugadores actúan de manera óptima, el resultado esperado es empate. Por lo tanto, el valor del estado inicial bajo juego perfecto es:

$$V(s_0) = 0$$

Esto significa que ningún jugador puede forzar una victoria si el otro responde óptimamente. En términos de equilibrio, existe una estrategia óptima para cada jugador tal que ninguno puede mejorar unilateralmente su resultado esperado. Ese perfil de estrategias constituye un equilibrio de Nash en estrategias puras para el juego extensivo: frente a juego óptimo del rival, desviarse no permite obtener una recompensa mayor.

3.2. Nocca Nocca

Nocca Nocca es un juego de tablero alternado para dos jugadores. Cada jugador controla piezas sobre una grilla y puede moverlas de acuerdo con reglas de legalidad definidas por el tablero. El ambiente incorpora control de turnos, acciones legales, detección de finalización y recompensas según el ganador.

Este ambiente tiene un espacio de acciones más amplio que Tic-Tac-Toe y permite estudiar el comportamiento de algoritmos de búsqueda en un juego con mayor complejidad combinatoria.

3.3. Kuhn Poker

Kuhn Poker es un juego de cartas simplificado de dos jugadores y suma cero [2]. Se utiliza una baraja de tres cartas y cada jugador recibe una carta privada. Los jugadores alternan acciones de pasar o apostar, y el resultado depende del historial de apuestas y de las cartas privadas.

El valor del ambiente radica en que es un juego pequeño pero con información imperfecta. Cada agente conoce su propia carta y el historial público, pero no conoce la carta del oponente. Por este motivo, Kuhn Poker es especialmente adecuado para evaluar ISMCTS y CFR.

4. Implementación

4.1. Interfaz de Juegos Alternados

Los ambientes del obligatorio comparten una interfaz base llamada **AlternatingGame**. Esta abstracción define operaciones comunes como **reset**, **step**, **observe**, **available_actions**, **reward**, **terminated**, **truncated**, **game_over** y **clone**.

La operación **clone** es central para los algoritmos de búsqueda, ya que permite simular acciones futuras sin modificar el estado real del juego. Esta separación entre estado real y estado simulado es utilizada por Minimax, MCTS e ISMCTS.

4.2. Agente Aleatorio

El agente aleatorio selecciona uniformemente una acción entre las acciones legales disponibles. Se utiliza como línea base para comparar el desempeño de agentes más sofisticados.

4.3. Minimax

El agente Minimax recorre el árbol de juego desde el estado actual. Cuando el turno pertenece al agente propio, selecciona la acción que maximiza el valor esperado; cuando actúa el oponente, asume que este minimizará dicho valor. La implementación permite limitar la profundidad y utilizar una función de evaluación cuando no se alcanza un estado terminal.

4.4. MCTS

El agente MCTS construye un árbol de búsqueda a partir del estado actual. En cada simulación selecciona nodos mediante UCT, expande acciones no exploradas, ejecuta rollouts aleatorios y retropropaga las recompensas obtenidas. La acción final se elige a partir del valor medio estimado de los hijos de la raíz.

4.5. ISMCTS

El agente ISMCTS extiende la lógica de MCTS para trabajar con estados de información. Para cada simulación se obtiene un estado compatible con lo observado por el agente. Si el ambiente implementa una forma específica de samplear desde el conjunto de información, el agente la utiliza; en caso contrario, recurre a una copia del estado actual.

4.6. CFR

El agente CFR mantiene nodos asociados a observaciones o estados de información. En cada iteración recorre recursivamente el juego, calcula utilidades por acción y actualiza regrets contrafactuales. La política aprendida surge del promedio de las políticas generadas por regret matching.

5. Experimentos

5.1. Tic-Tac-Toe

En Tic-Tac-Toe se evaluaron MiniMax y Monte Carlo Tree Search (MCTS). Como es un juego pequeño y de información perfecta, permite comparar directamente una búsqueda por profundidad con una búsqueda basada en simulaciones.

El resultado esperado es empate, por eso, en los experimentos se toma el empate como referencia de equilibrio y analizamos cómo cambian los resultados al variar la profundidad de MiniMax, las simulaciones de MCTS y la cantidad de rollouts.

MiniMax vs MiniMax

El este primer bloque enfrentamos MiniMax contra MiniMax usando la misma profundidad para ambos agentes, variando `depth` de 1 a 9. Cada configuración se ejecutó con 3 partidas por lado (3: X y 3:O), es decir, 6 partidas por profundidad, esto lo hicimos para asegurarnos que no exista ventaja por quien comienza. Sabemos que en profundidades bajas, MiniMax debe cortar la búsqueda y usar la función heurística del ambiente, mientras que en profundidad 9 alcanza el árbol completo y representa juego óptimo.

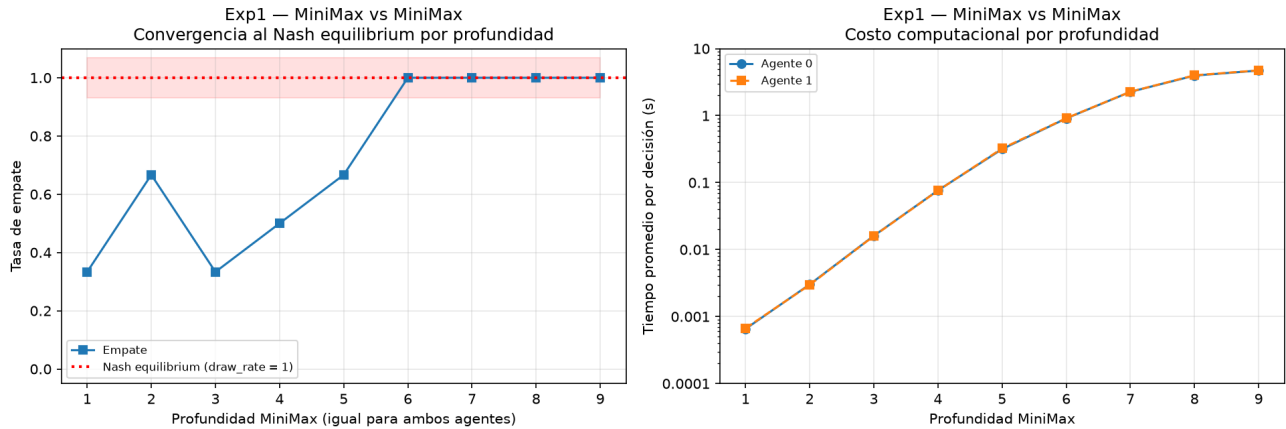


Figura 1: MiniMax vs MiniMax variando la profundidad.

Los resultados muestran que la tasa de empate aumenta al profundizar la búsqueda. Desde profundidad 6 en adelante todas las partidas terminaron en empate. Este es el comportamiento esperado, cuando ambos agentes tienen una búsqueda suficientemente profunda, ninguno logra forzar una victoria.

Por otro lado, se observa que el costo computacional crece rápidamente con la profundidad. El tiempo promedio de decisión pasa de menos de 1ms en profundidad 1 a aproximadamente 4.7s en profundidad 9. Por este motivo, para Tic-Tac-Toe no siempre resulta necesario usar la profundidad completa, como vemos con profundidades intermedias, como profundidad 6 en esta corrida, ya alcanzan el equilibrio y reducen de forma importante el gasto computacional.

MCTS vs MCTS: simulaciones

El segundo bloque enfrenta MCTS contra MCTS usando el mismo presupuesto para ambos jugadores, con `rollouts=10` fijo y variando las simulaciones en `[10, 25, 50, 100, 200, 300, 500]`. En cada punto se jugaron 50 partidas por lado, para un total de 100 partidas por configuración.

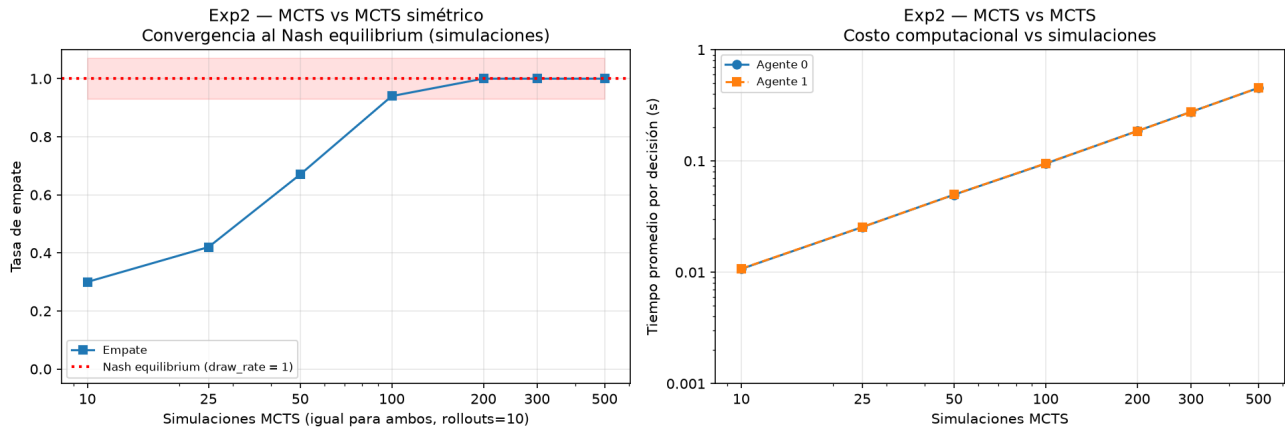


Figura 2: MCTS vs MCTS variando la cantidad de simulaciones.

La tasa de empate aumenta claramente con el número de simulaciones. Con 10 simulaciones fue 30 %, con 50 simulaciones subió a 67 %, con 100 simulaciones llegó a 94 %, y desde 200 simulaciones

todas las partidas terminaron en empate. Esto muestra una convergencia empírica hacia el equilibrio de Nash cuando ambos agentes MCTS disponen de suficiente simulaciones.

El costo computacional también aumenta de forma clara con el número de simulaciones, aproximadamente 11ms por decisión con 10 simulaciones y 1s con 500 simulaciones. Esto confirma que el presupuesto de simulaciones afecta directamente el tiempo de decisión de MCTS.

MCTS vs MCTS: rollouts

En este bloque analizamos el efecto de los rollouts usando dos presupuestos fijos de simulaciones: `simulations=50` y `simulations=100`. Para cada presupuesto se varía la cantidad de rollouts en `[1, 2, 5, 10, 15, 20, 30]`. Igual que antes, ambos agentes usan el mismo presupuesto y el agente 0 juega 80 partidas como X y 80 como O, para un total de 160 partidas por configuración.

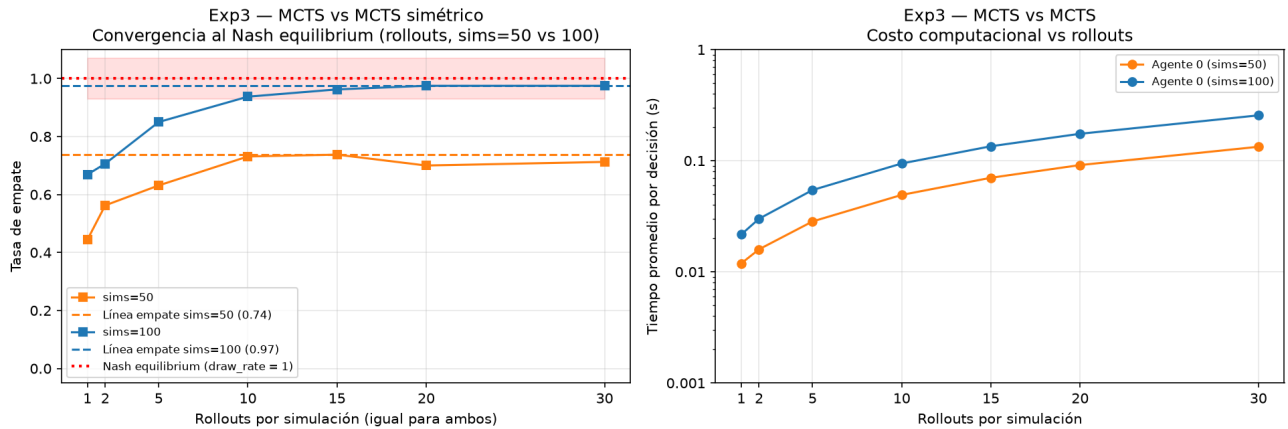


Figura 3: MCTS vs MCTS variando rollouts para dos presupuestos de simulaciones.

Los resultados muestran que aumentar rollouts ayuda a estabilizar la estimación de MCTS, pero no alcanza por sí solo. Con 50 simulaciones, la tasa de empate sube de 44 % (1 rollout) a 74 % con 15 rollouts, pero luego se estanca y aunque se aumenten los rollouts el agente no logra llegar al equilibrio. En cambio, con 100 simulaciones el comportamiento se acerca mucho más al valor esperado del juego, donde la tasa de empate pasa de 67 % (1 rollout) a 85 % con 5 rollouts, 93.75 % con 10

rollouts y 97.5 % con 20 y 30 rollouts.

Esto indica que los rollouts reducen la varianza de las evaluaciones, porque cada nodo se estima promediando más partidas simuladas, pero no reemplazan la exploración del árbol. Si se hacen pocas simulaciones, MCTS puede evaluar mejor las acciones que visitó, pero sigue visitando una parte limitada del árbol. Además, el costo crece con ambos parámetros: para 100 simulaciones, el tiempo promedio pasa de aproximadamente 21.7ms por decisión con 1 rollout a 255.8ms con 30 rollouts.

MCTS vs MiniMax

En este bloque comparamos MCTS contra MiniMax. El agente 0 es MCTS, con simulaciones en [50, 100, 200] y `rollouts=12`; el agente 1 es MiniMax, con profundidades de 1 a 7. Para cada combinación se jugaron 20 partidas por lado.

Simulaciones MCTS	Profundidad MiniMax	Gana MCTS	Empates	Gana MiniMax	Reward prom. MCTS
50	1	34	6	0	0.850
50	2	0	37	3	-0.075
50	3	7	29	4	0.075
50	4	10	26	4	0.150
50	5	12	20	8	0.100
50	6	0	33	7	-0.175
50	7	0	30	10	-0.250
100	1	27	12	1	0.650
100	2	0	40	0	0.000
100	3	11	27	2	0.225
100	4	9	31	0	0.225
100	5	10	30	0	0.250
100	6	0	40	0	0.000
100	7	0	39	1	-0.025
200	1	31	9	0	0.775
200	2	0	40	0	0.000
200	3	7	33	0	0.175
200	4	9	31	0	0.225
200	5	4	36	0	0.100
200	6	0	40	0	0.000
200	7	0	40	0	0.000

Tabla 1: Resultados de MCTS vs MiniMax. Cada fila resume 40 partidas desde la perspectiva de MCTS.

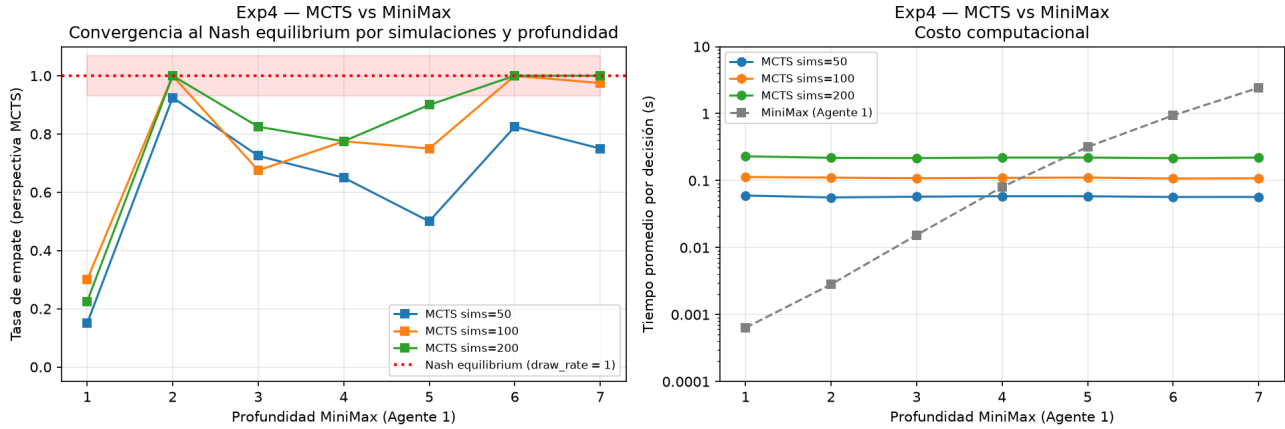


Figura 4: MCTS vs MiniMax variando simulaciones y profundidad.

Contra MiniMax de profundidad 1, MCTS obtiene una ventaja clara: gana 85 % de las partidas con 50 simulaciones, 67.5 % con 100 simulaciones y 77.5 % con 200 simulaciones. Contra profundidad 2, el resultado se acerca al equilibrio, con 100 y 200 simulaciones las 40 partidas terminan empatadas, mientras que con 50 simulaciones aparecen 37 empates y 3 derrotas de MCTS. Contra profundidades 3 a 5, MCTS obtiene resultados positivos en todas las configuraciones, aunque con variación por la naturaleza estocástica de los rollouts, ya que estos completan partidas simuladas de forma aleatoria y, por lo tanto, el valor estimado de una acción puede cambiar levemente entre corridas, aun partiendo de una posición similar. Contra profundidades 6 y 7, MCTS no consigue victorias. Con 100 y 200 simulaciones logra empates en el 99 % de las veces. Con 50 simulaciones, en cambio, todavía aparecen derrotas frente a las búsquedas más profundas.

El gráfico de tiempos confirma los costos de ambos algoritmos. Para MiniMax, pasar de profundidad 1 a profundidad 7 implica pasar de menos de 1ms a más de 2 s por decisión, incluso en un juego tan pequeño como Tic-Tac-Toe. En MCTS, el costo depende principalmente de las simulaciones, en esta corrida estuvo alrededor de 0.06 s con 50 simulaciones, 0.11 s con 100 simulaciones y 0.22 s con 200 simulaciones. Podemos ver como el aumento de simulaciones en MCTS no afecta tanto el costo computacional, como si lo hace el aumento de profundidad en minimax. Además observamos

que MCTS consigue mas partidas ganadas que MiniMax.

MCTS vs MiniMax óptimo

Como contraste final se enfrentó MCTS con 200 simulaciones y 10 rollouts contra MiniMax con profundidad 9. Se jugaron 3 partidas con MCTS como X y 3 como O, debido al costo de la búsqueda completa sin poda alfa-beta.

Las 6 partidas terminaron en empate: 0 % victorias de MCTS, 100 % empates y 0 % derrotas. Este resultado coincide con el valor minimax de Tic-Tac-Toe. Contra un oponente óptimo no se puede forzar una victoria, pero con presupuesto suficiente MCTS logró sostener el empate.

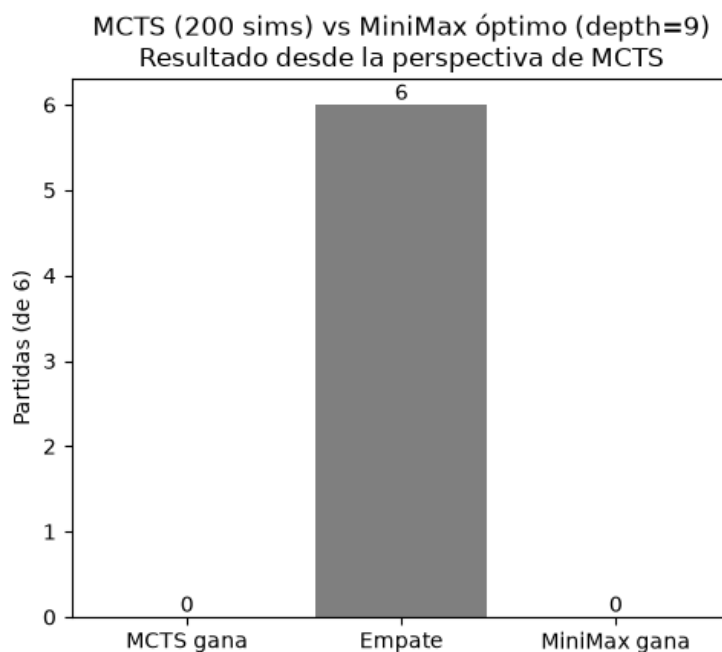


Figura 5: MCTS contra MiniMax óptimo con profundidad completa.

Benchmark de tiempos

Además de medir desempeño en partidas, se comparó el tiempo promedio de decisión de los agentes. Para que el benchmark fuera comparable, todos los agentes jugaron la misma cantidad de

partidas como 0 contra Random como X.

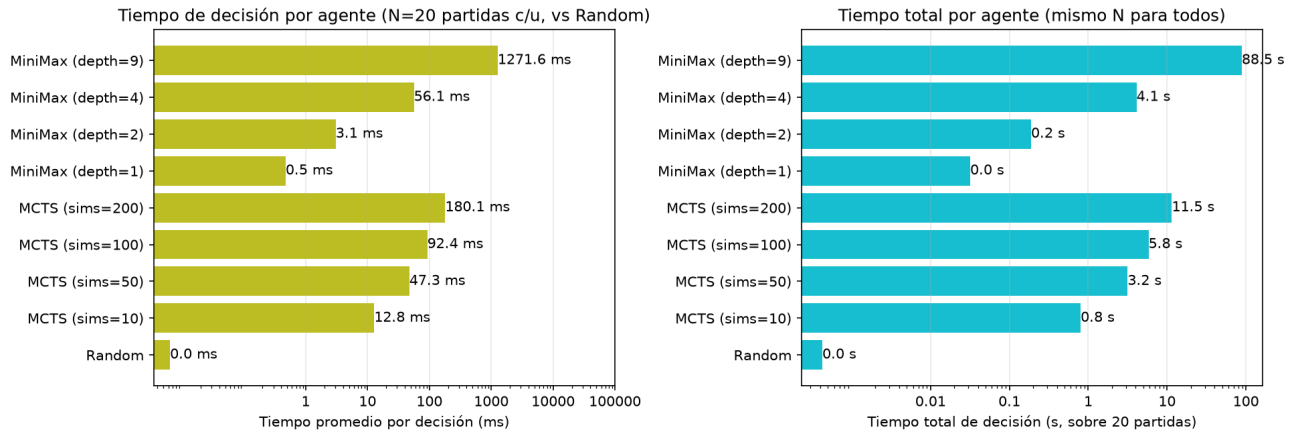


Figura 6: Comparación de tiempos de decisión entre agentes.

El tiempo promedio por decisión de MiniMax pasó de aproximadamente 0.00047 s con profundidad 1 a 1.272 s con profundidad 9, un aumento de varios órdenes de magnitud. MCTS, en cambio, escaló de forma más suave, de 0.0128 s con 10 simulaciones a 0.1801 s con 200 simulaciones. En tiempo total sobre 20 partidas, MiniMax con profundidad 9 insumió alrededor de 88.5 s, mientras que MCTS con 200 simulaciones tomó alrededor de 11.5 s.

Conclusiones de Tic-Tac-Toe

Los experimentos muestran que Tic-Tac-Toe funciona como un ambiente de validación adecuado para MiniMax y MCTS. MiniMax permite representar desde un agente heurístico con profundidad limitada hasta un jugador óptimo con profundidad completa. En self-play, MiniMax converge claramente al empate al aumentar la profundidad, y MCTS también converge al empate cuando se aumenta el presupuesto de simulaciones o se usan suficientes rollouts.

La comparación con MiniMax ilustra el compromiso central entre búsqueda exhaustiva y búsqueda por muestreo. En Tic-Tac-Toe, MiniMax con profundidad completa es posible y permite recuperar exactamente el equilibrio de Nash del juego. Sin embargo, su costo crece rápidamente

con la profundidad. MCTS no garantiza exactitud en cada decisión y sus resultados dependen de la varianza de las simulaciones, dado que los rollouts completan partidas de forma aleatoria, distintas simulaciones pueden producir estimaciones algo diferentes para una misma posición. Aun así, MCTS permite regular el costo computacional mediante **simulations** y **rollouts**. Esta diferencia es la razón por la que MCTS se vuelve especialmente atractivo en juegos con árboles más grandes, donde una búsqueda exhaustiva deja de ser viable.

5.2. Nocca Nocca

En Nocca Nocca se evaluaron principalmente MiniMax y Monte Carlo Tree Search (MCTS). A diferencia de Tic-Tac-Toe, este juego tiene un tablero más grande, más acciones legales por estado y partidas potencialmente más largas. Por eso no se usa una solución exacta del juego como referencia, sino que se comparan resultados empíricos y tiempos de decisión. Además, los experimentos se ejecutan con pocas partidas, porque aun con pocos juegos algunas configuraciones tardan horas en terminar.

En los análisis, el equilibrio se representa mediante el acercamiento a una recompensa promedio de 0, donde valores positivos favorecen al agente evaluado y valores negativos favorecen al rival, por lo tanto la convergencia a cero significa que los agentes ganas en un 50 % de las veces.

MiniMax vs MiniMax

Se evaluó MiniMax contra sí mismo usando profundidades 1, 2, 3 y 4. En cada configuración se jugaron 3 partidas por cada rol, para un total de 6 partidas por profundidad. A diferencia de Tic-Tac-Toe, MiniMax no puede buscar hasta el final de la partida. Además, la función de evaluación utilizada devuelve recompensa terminal cuando el juego termina y 0 en estados no terminales. Por eso, con profundidades acotadas, MiniMax solo distingue con claridad posiciones donde aparece una victoria o derrota dentro del horizonte de búsqueda.

Profundidad	Victorias A0	Empates	Victorias A1	Recompensa A0	Tiempo A0 (s)
1	2	1	3	-0.17	0.0007
2	1	4	1	0.00	0.0141
3	4	1	1	0.50	0.3729
4	0	6	0	0.00	7.6805

Tabla 2: Resultados de MiniMax vs MiniMax en Nocca Nocca.

Los resultados no muestran una mejora monótona al aumentar la profundidad. Con profundidad 2 y 4 la recompensa promedio del agente 0 queda en 0, mientras que con profundidad 3 aparece una ventaja para el agente 0 en esta corrida. En profundidad 4 todas las partidas terminaron con recompensa 0, es decir que el juego llegó al límite de pasos (`max_steps`). Mirar más jugadas hacia adelante no alcanzó para conseguir más victorias dentro de ese límite. Esto puede pasar porque el juego tiene muchas acciones posibles y la evaluación de estados intermedios aporta poca información. En cambio, el costo computacional sí crece mucho, pasando de menos de una milésima de segundo por decisión con profundidad 1 a más de 7 segundos por decisión con profundidad 4.

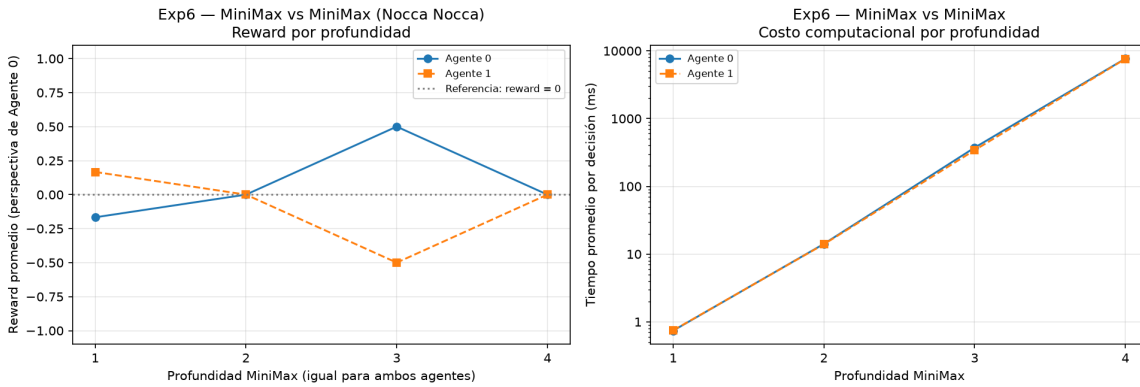


Figura 7: MiniMax vs MiniMax con evaluación simétrica.

MCTS vs MCTS

Para analizar MCTS en self-play se realizaron dos casos, en el primero se varió la cantidad de simulaciones, manteniendo 5 rollouts por acción. Se probaron 25, 50, 100, 200 y 500 simulaciones, con 10 partidas por rol en cada configuración.

Simulaciones	Victorias A0	Empates	Victorias A1	Recompensa A0	Tiempo A0 (s)
25	8	0	12	-0.20	1.451
50	12	0	8	0.20	3.089
100	10	0	10	0.00	6.252
200	11	0	9	0.10	12.296
500	10	0	10	0.00	30.683

Tabla 3: MCTS vs MCTS variando simulaciones.

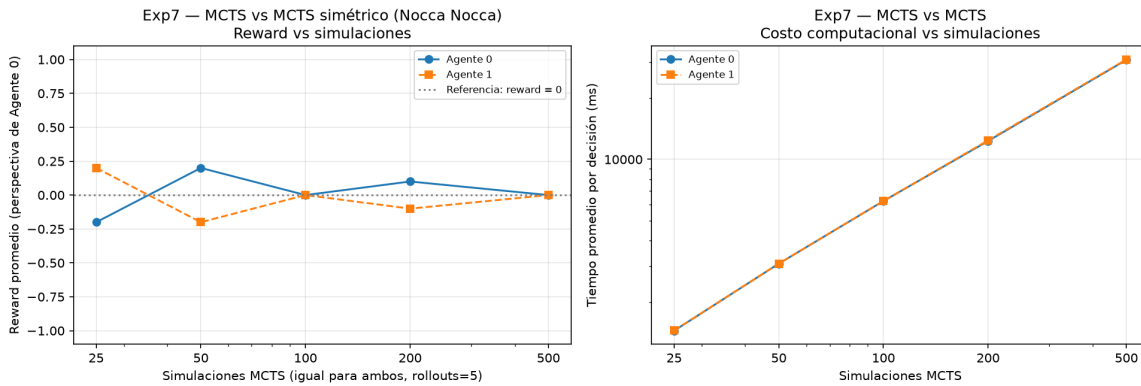


Figura 8: MCTS vs MCTS variando la cantidad de simulaciones.

Se observa que la recompensa promedio oscila alrededor de 0, lo que indica que ningún rol domina de manera estable en esta corrida. A partir de 100 simulaciones, los valores quedan más cerca de 0, lo que muestra que se acercan al equilibrio. El resultado más relevante es el crecimiento del tiempo de decisión, pasar de 25 a 500 simulaciones aumenta el costo desde aproximadamente 1.451s

hasta 30.683s por decisión. En este juego, por lo tanto, aumentar simulaciones mejora la exploración, pero rápidamente vuelve costosa cada jugada.

En el segundo caso se fijaron las simulaciones en 100 simulaciones y se varió la cantidad de rollouts por simulación. Se probaron 1, 5, 10 y 20 rollouts. El objetivo de este experimento es observar si, manteniendo baja la cantidad de simulaciones, aumentar los rollouts alcanza para acercar el self-play al equilibrio, representado por una recompensa promedio cercana a 0.

Rollouts	Victorias A0	Empates	Victorias A1	Recompensa A0	Tiempo A0 (s)
1	8	0	12	-0.20	1.134
5	15	0	5	0.50	6.040
10	9	0	11	-0.10	12.816
20	12	0	8	0.20	25.610

Tabla 4: MCTS vs MCTS variando rollouts con 100 simulaciones fijas.

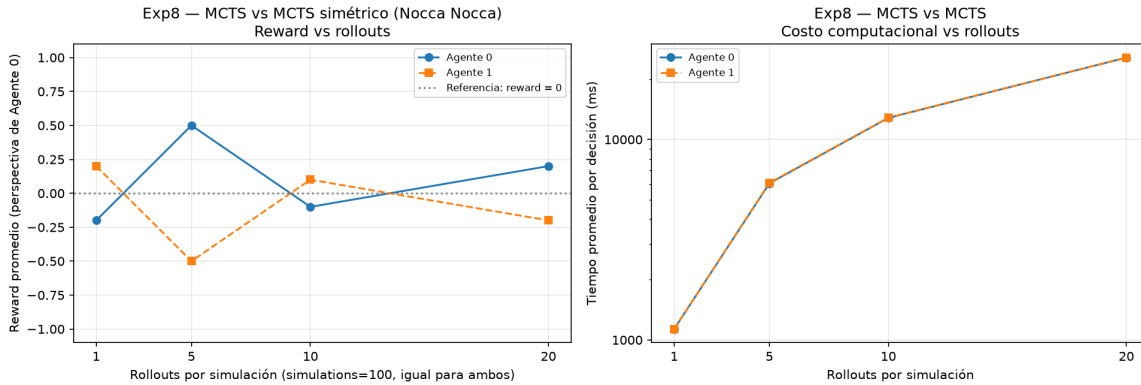


Figura 9: MCTS vs MCTS variando rollouts con bajo presupuesto de simulaciones.

Los resultados muestran que esto no ocurre de forma clara. Aunque más rollouts permiten promediar más trayectorias aleatorias desde cada nodo, no compensan completamente la falta de exploración del árbol, dado el número de simulaciones es limitado. Se observa que la recompensa

promedio no converge de manera monótona hacia 0. Los mejores resultados aparecen alrededor de 10 rollouts, donde la recompensa queda más cerca del equilibrio. Al aumentar a 20 rollouts no se obtiene una mejora clara, lo que sugiere que, aumentar rollouts mejora la estimación local de acciones ya exploradas, pero no reemplaza la necesidad de contar con suficientes simulaciones para explorar mejor el árbol.

MCTS vs MiniMax

También se comparó MCTS contra MiniMax. La comparación permite observar dos formas distintas de jugar, donde MCTS aumenta el número de simulaciones y MiniMax aumenta la profundidad de búsqueda.

En la configuración actual de la notebook se evaluó MCTS con 100 y 200 simulaciones, usando 5 rollouts por simulación, contra MiniMax con profundidades 2, 3 y 4. En cada combinación se jugaron 8 partidas, combinando partidas con MCTS como primer y segundo jugador.

Simulaciones	Prof. MiniMax	Victorias MCTS	Empates	Victorias MiniMax	Recompensa MCTS	Tiempo MCTS (s)	Tiempo MiniMax (s)
100	2	5	0	3	0.25	0.506	0.012
100	3	7	0	1	0.75	0.583	0.279
100	4	3	0	5	-0.25	0.438	5.363
200	2	8	0	0	1.00	0.921	0.014
200	3	7	0	1	0.75	0.974	0.260
200	4	4	0	4	0.00	0.960	6.057

Tabla 5: Resultados de MCTS vs MiniMax en Nocca Nocca. Cada fila resume 8 partidas desde la perspectiva de MCTS.

Los resultados muestran que MCTS obtiene ventaja clara contra MiniMax de profundidad 2 y 3, especialmente con 200 simulaciones. Sin embargo, contra MiniMax de profundidad 4, MCTS con 100 simulaciones queda por debajo. Es con MiniMax de profundidad 4 y MCTS con 200 simulaciones donde el resultado queda equilibrado.

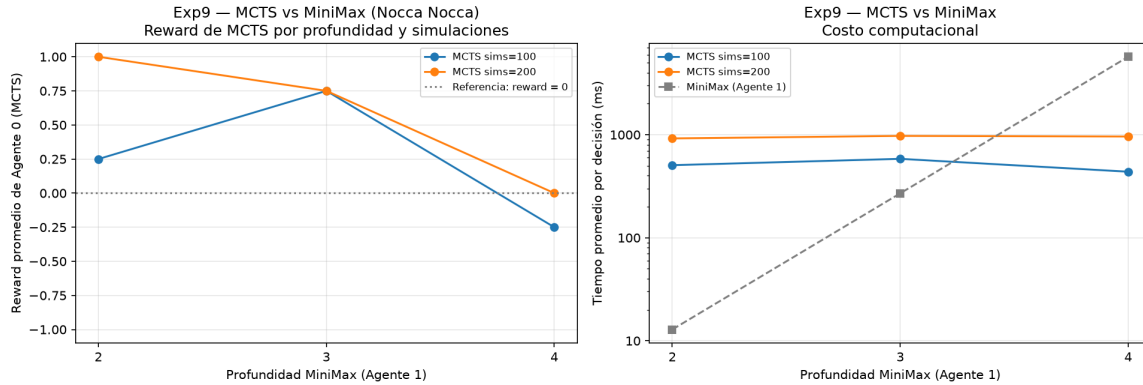


Figura 10: MCTS vs MiniMax variando simulaciones de MCTS y profundidad de MiniMax.

En las gráficas no solo se observa que MCTS con 200 simulaciones queda equilibrado contra MiniMax con profundidad 4, sino también cómo el aumento de profundidad en MiniMax y el aumento de simulaciones en MCTS afectan el costo computacional. En particular, MCTS con 200 simulaciones tiene un costo bastante menor que MiniMax con profundidad 4, aproximadamente 1 s contra 6 s por decisión, y aun así logra un resultado equilibrado. Por eso, en esta comparación resulta una opción más conveniente.

Conclusiones de Nocca Nocca

Nocca Nocca muestra con más claridad que Tic-Tac-Toe el costo de buscar en juegos con muchas acciones posibles. En este caso no alcanza con aumentar la profundidad o el número de rollouts sin mirar el costo, incluso con pocas partidas, algunas configuraciones tardaron mucho tiempo en terminar o directamente no terminaron.

En MiniMax vs MiniMax, aumentar la profundidad no produjo una mejora clara, con profundidad 4 llegó a recompensa promedio 0, lo que significa que el juego termino porque alcanzó el numero maximo de steps, ademas de tener un costo muy alto. Esto se debe a que la evaluación de estados intermedios aporta poca información y el árbol de búsqueda crece rápidamente.

En MCTS vs MCTS, aumentar simulaciones ayudó a acercarse a recompensa 0, mientras que

aumentar rollouts tuvo un efecto más limitado, donde alrededor de 10 rollouts se obtuvieron los mejores resultados, pero seguir aumentando no mejoró claramente.

La comparación MCTS vs MiniMax resume esta diferencia. MCTS con 200 simulaciones logró quedar equilibrado contra MiniMax con profundidad 4, pero con un tiempo promedio mucho menor. Por eso, para Nocca Nocca, MCTS resulta más flexible que MiniMax con profundidad fija, especialmente cuando no se cuenta con una heurística fuerte para evaluar estados no terminales.

5.3. Leduc Hold'em

Leduc Hold'em se utiliza para evaluar agentes en un juego de suma cero con información imperfecta. Cada jugador recibe una carta privada, luego se revela una carta pública y las decisiones de apuesta se toman sin conocer la carta del rival. A diferencia de Tic-Tac-Toe y Nocca Nocca, el estado observado por un agente no determina completamente el estado real del juego.

Como criterio empírico, una reward promedio estable y cercana a 0 en enfrentamientos simétricos sugiere que ninguno de los agentes obtiene una ventaja persistente. Sin embargo, esto no prueba por sí solo que se alcanzó el equilibrio de Nash.

En esta sección se comparan Counterfactual Regret Minimization (CFR) e Information Set Monte Carlo Tree Search (ISMCTS). CFR aprende una política promedio sobre estados de información mediante regret matching, mientras que ISMCTS realiza búsquedas sobre determinaciones compatibles con la observación del agente. Las recompensas se reportan desde la perspectiva de `player_0`; valores positivos favorecen a `player_0` y valores negativos favorecen a `player_1`.

CFR

Primero se evaluó CFR contra un agente Random. Esta comparación mide el reward promedio de la política aprendida contra un oponente aleatorio, por lo que sirve para verificar que CFR aprende algo útil y logra explotar decisiones no estratégicas. Sin embargo, no es una medida de convergencia

al equilibrio, porque un buen desempeño contra Random no implica que la política sea difícil de explotar por un rival óptimo. En la corrida guardada, CFR como `player_0` obtuvo una recompensa promedio de 0.581 contra Random en 2000 partidas.

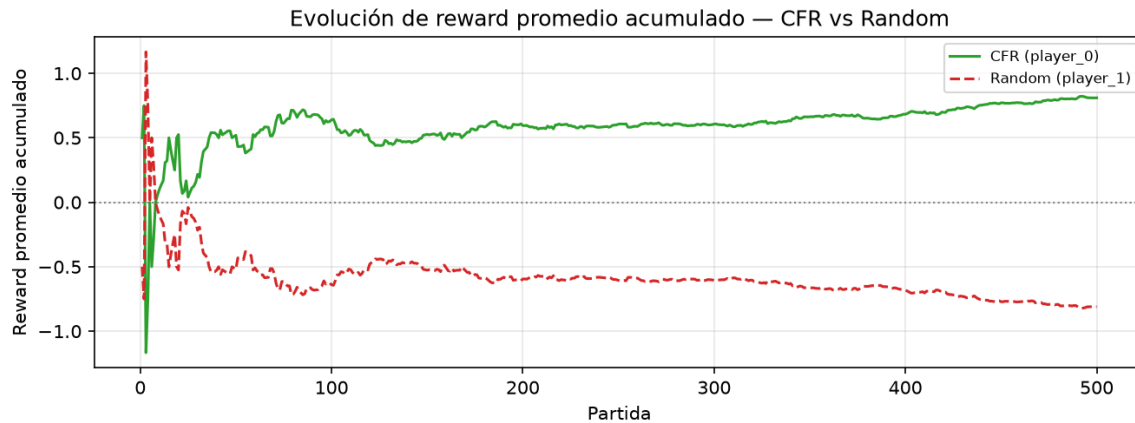


Figura 11: Evolución del reward promedio acumulado en CFR vs Random.

Para analizar la estabilidad de la estrategia aprendida se enfrentaron dos agentes CFR entre sí. Si la estrategia converge hacia un equilibrio de Nash, entonces dos jugadores con el mismo entrenamiento no deberían obtener una ventaja persistente, y la reward promedio esperada debería quedar cerca de 0 por simetría. En la corrida guardada, CFR contra CFR obtuvo una recompensa promedio de -0.040 para `player_0` en 2000 partidas.

La evolución del reward promedio acumulado en CFR vs CFR se mantiene alrededor de 0 con oscilaciones propias de la varianza del juego, en lugar de mostrar una tendencia sostenida a favor de uno de los jugadores. Por eso, esta comparación se usa como criterio empírico más cercano para estudiar convergencia al equilibrio.

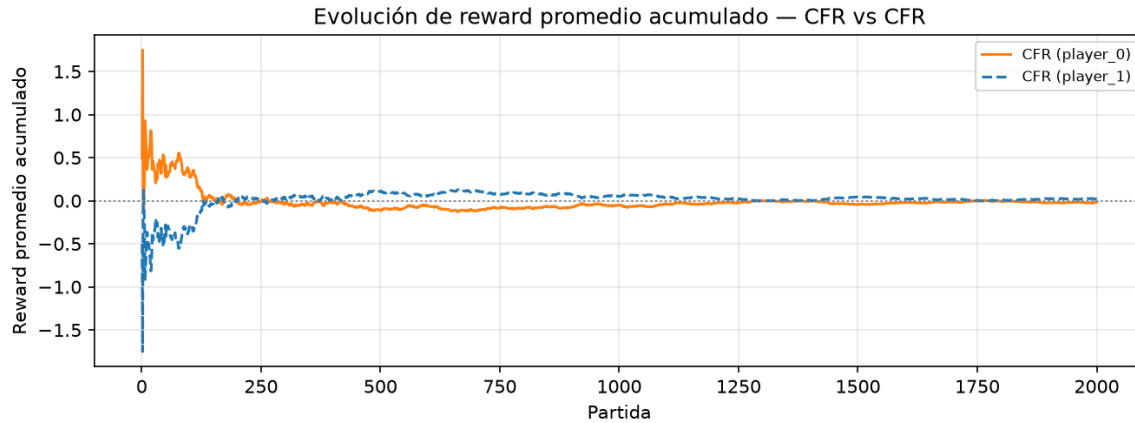


Figura 12: Evolución del reward promedio acumulado en CFR vs CFR.

ISMCTS

Luego se evaluó ISMCTS, usando determinización desde el conjunto de información del agente. En cada simulación, el ambiente samplea una carta posible para el oponente compatible con la carta propia y el historial público. Con 50 simulaciones y 5 rollouts, ISMCTS obtuvo una recompensa promedio de 0.942 contra Random en 200 partidas.

También se enfrentaron agentes ISMCTS entre sí para aislar el efecto de los hiperparámetros. Al variar las simulaciones en $[10, 25, 50, 100, 200]$ con 5 rollouts fijos, la recompensa promedio de `player_0` se mantuvo cerca de 0: 0.113, -0.010, -0.063, -0.078 y -0.081 respectivamente. Esto indica que, cuando ambos jugadores tienen el mismo presupuesto, ninguno domina de forma estable. El costo sí crece casi linealmente, desde aproximadamente 42.7 ms por decisión con 10 simulaciones hasta 798.2 ms con 200 simulaciones.

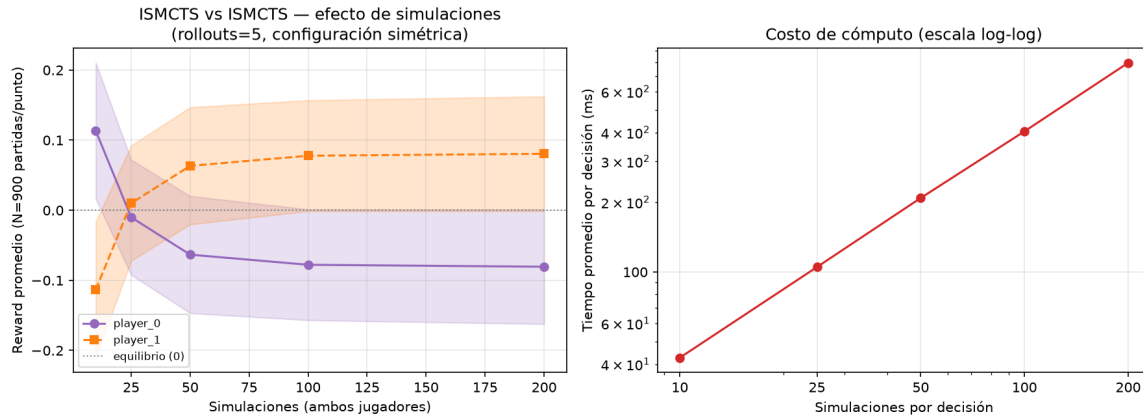


Figura 13: ISMCTS vs ISMCTS variando simulaciones.

Al variar la cantidad de rollouts con 50 simulaciones fijas, los resultados fueron más ruidosos. Los rewards promedio oscilaron entre -0.317 y 0.260, sin una mejora monótona. Esto muestra que, en Leduc, aumentar rollouts reduce varianza en cada evaluación local, pero no garantiza por sí solo una ventaja estable, especialmente por la incertidumbre de la carta oculta y la varianza natural de las apuestas.

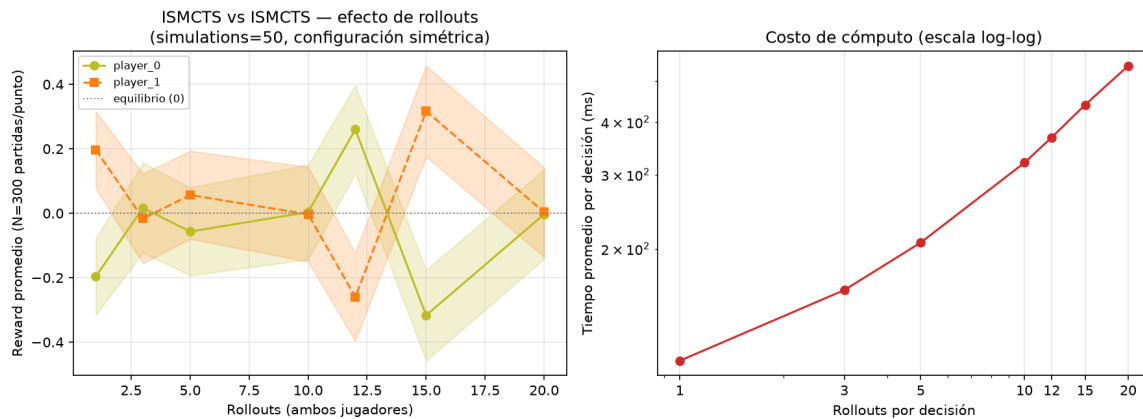


Figura 14: ISMCTS vs ISMCTS variando rollouts.

CFR vs ISMCTS

Finalmente se compararon CFR e ISMCTS directamente. Cuando CFR juega como `player_0` contra ISMCTS con 50 simulaciones y 5 rollouts, la recompensa promedio de `player_0` fue -0.382 en 200 partidas. Esto indica que, en esa corrida, ISMCTS obtuvo ventaja desde la posición de `player_1`. De todos modos, la comparación es sensible a la posición, al número de partidas y al presupuesto de simulaciones, por lo que se reporta como una comparación empírica y no como una solución exacta del juego.

El resumen de medias muestra la tendencia general de los enfrentamientos: CFR e ISMCTS superan claramente a Random, CFR contra CFR queda cerca de 0, e ISMCTS contra CFR puede variar según la posición y el presupuesto. En términos de tiempo, CFR es muy barato en inferencia una vez entrenado, porque solo consulta la política aprendida. En el benchmark, CFR con 200 iteraciones tomó aproximadamente 0.004 ms por decisión, mientras que ISMCTS creció de 42.985 ms con 10 simulaciones a 404.465 ms con 100 simulaciones.

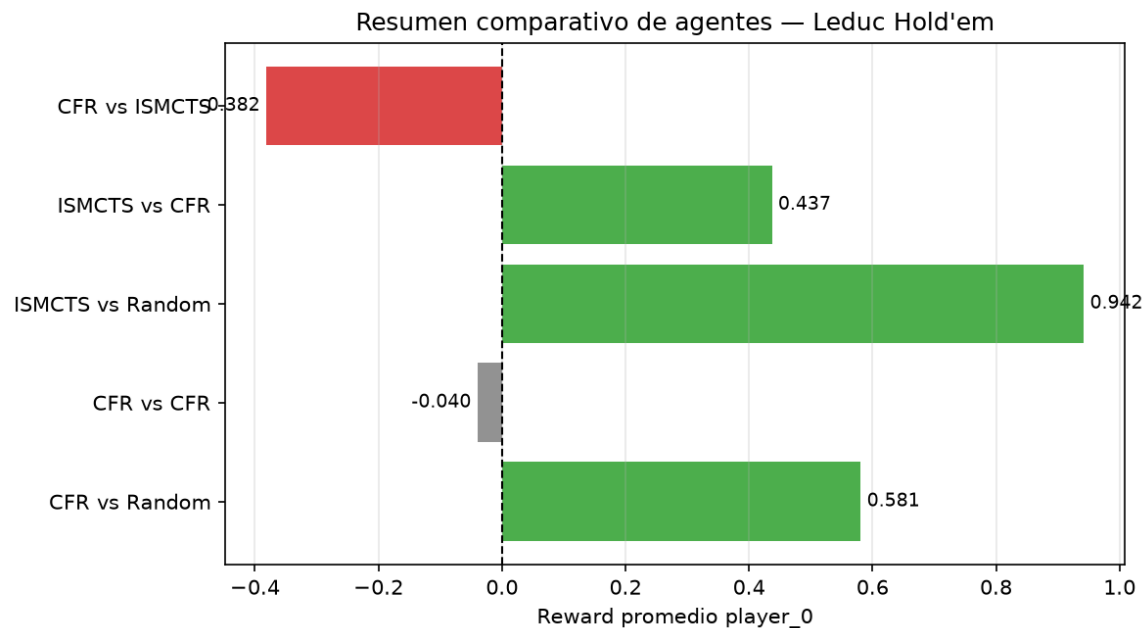


Figura 15: Resumen comparativo de reward promedio en Leduc.

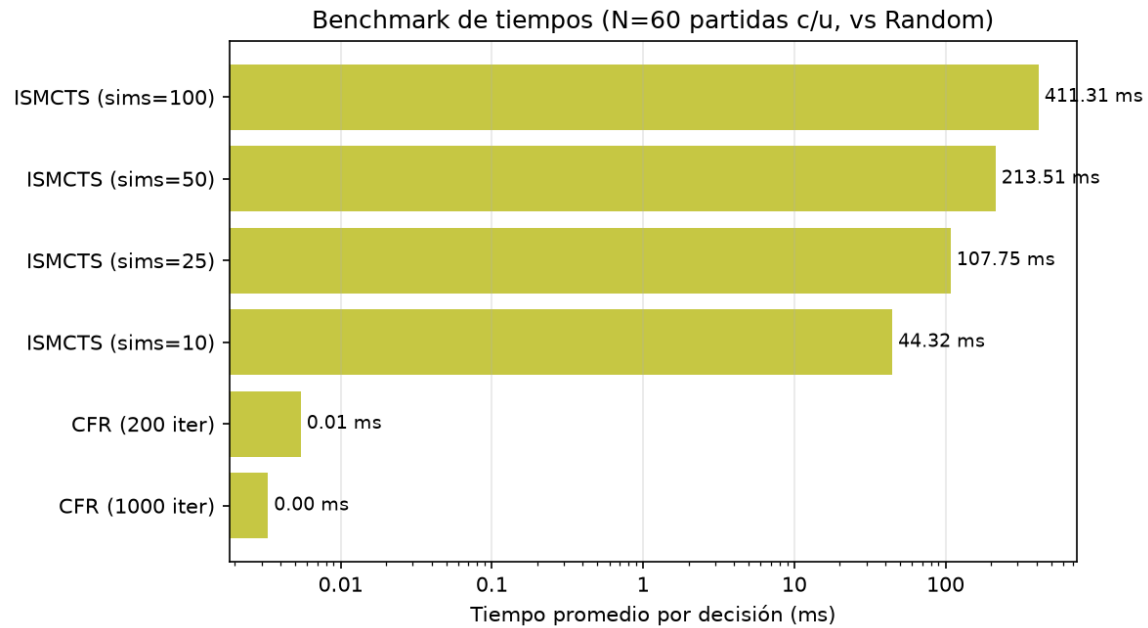


Figura 16: Benchmark de tiempo promedio por decisión en Leduc.

Conclusiones de Leduc

Los experimentos de Leduc muestran la diferencia entre aprender una política para estados de información y planificar mediante simulaciones online. CFR requiere entrenamiento previo, pero luego decide casi instantáneamente y tiende a estabilizarse cuando se enfrenta a otra política CFR. ISMCTS no necesita entrenamiento offline, pero paga un costo por decisión que crece con las simulaciones y los rollouts. En un juego de información imperfecta, esta diferencia es central: no alcanza con buscar sobre un estado completamente visible, sino que el agente debe razonar sobre los estados compatibles con lo que observa.

5.4. Kuhn Poker

Random vs Random

CFR vs Random

ISMCTS vs Random

CFR vs ISMCTS

Política aprendida por CFR

Análisis de información imperfecta

6. Conclusión

Este obligatorio permite estudiar problemas de decisión secuencial multiagente mediante juegos alternados. A partir de una interfaz común para ambientes, se implementaron agentes representativos de distintas familias: búsqueda adversarial exhaustiva, búsqueda Monte Carlo, búsqueda sobre estados de información y minimización de regret.

Tic-Tac-Toe ofrece un punto de partida controlado para verificar el funcionamiento de los algoritmos en información perfecta. Nocca Nocca introduce un espacio de acciones más complejo, donde la eficiencia de la búsqueda se vuelve más relevante. Leduc Hold'em y Kuhn Poker agregan información imperfecta y permiten observar la necesidad de razonar sobre estados compatibles con la observación del agente.

En conjunto, los ambientes y agentes implementados muestran cómo la estructura del juego condiciona la elección del algoritmo. Minimax resulta natural para juegos pequeños de información perfecta, MCTS permite escalar mediante simulaciones, ISMCTS adapta la búsqueda a información imperfecta y CFR permite aprender políticas promedio en juegos extensivos.

Referencias

- [1] Levente Kocsis and Csaba Szepesvari. Bandit based monte-carlo planning. *European Conference on Machine Learning*, 2006.
- [2] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*. Princeton University Press, 1950.
- [3] Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. Monte carlo sampling for regret minimization in extensive games. *Advances in Neural Information Processing Systems*, 2009.
- [4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 2021.

- [5] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.